

Cours complet : le protocole Agent2Agent (A2A)

Sep 26, 2026 · @Alex

Sommaire

- [1. Introduction : pourquoi A2A](#)
- [2. Concepts fondamentaux](#)
- [3. Architecture en couches et bindings](#)
- [4. Cycle de vie des tâches, streaming et notifications push](#)
- [4 bis. Résilience : crash serveur et coupure réseau](#)
- [5. Découverte des agents et Agent Card](#)
- [6. Sécurité : authentification, autorisation, confiance](#)
- [7. A2A face à MCP et aux autres protocoles](#)
- [8. Implémentation pas à pas avec le SDK Python](#)
- [9. Patterns d'architecture et bonnes pratiques de production](#)
- [10. Avantages, inconvénients et limites](#)
- [11. Écosystème, feuille de route et ressources](#)

1. Introduction : pourquoi A2A

A2A (Agent2Agent) est le standard ouvert qui permet à des agents IA opaques, construits par des équipes, des frameworks et des fournisseurs différents, de se découvrir, de se déléguer des tâches et d'échanger des résultats sans exposer leur fonctionnement interne. Ce cours s'appuie sur la **spécification v1.0.0**, première version stable, publiée le 12 mars 2026, et signale ce qui arrive en v1.1.

Le problème résolu

Sans protocole commun, chaque couple d'agents (un agent LangGraph qui appelle un agent CrewAI, un agent Salesforce qui délègue à un agent ServiceNow) exige une intégration sur mesure. Le coût croît en $N \times N$. A2A ramène ce coût à N : chaque agent expose une seule interface normalisée, décrite par une Agent Card.

Le coût d'intégration passe de $N \times N$ à N



intégration point à point contre protocole commun · 4 agents

Avec 4 agents, il faut déjà 6 intégrations spécifiques ; avec A2A, chaque agent n'expose qu'une interface, et tout nouvel agent est immédiatement joignable par les autres.

Le principe fondateur est **l'exécution opaque** : un agent collabore sur la base des capacités qu'il déclare et des informations échangées, jamais en partageant sa mémoire, ses outils, ses prompts ou son plan interne. C'est ce qui distingue A2A d'un simple appel d'outil.

Objectifs et principes de conception

Objectif	Traduction concrète dans le protocole
Interopérabilité	Modèle de données unique, trois bindings fonctionnellement équivalents
Collaboration	Délégation via Task, multi-tour via contextId et taskId
Découverte	Agent Card publiée à une URI bien connue
Flexibilité	Synchrone, streaming, notifications push
Sécurité	Réutilisation des standards web : TLS, OAuth 2.0, OIDC, mTLS
Asynchronisme natif	Tâches longues, humain dans la boucle, états interrompus

Les principes directeurs : réutiliser des standards éprouvés (HTTP, JSON-RPC 2.0, SSE, gRPC), viser l'entreprise dès le départ (authentification, traçabilité, supervision), être « async first » et agnostique vis-à-vis des modalités (texte, fichiers, données structurées, formulaires).

Historique et gouvernance

Date	Étape
27 août 2026	A2A accepté comme projet « Growth Stage » de l'Agentic AI Foundation (AAIF), aux côtés de MCP, goose et AGENTS.md
22 juillet 2026	SDK JavaScript 1.0.0 en disponibilité générale
12 mars 2026	Publication de la spécification v1.0.0, première version stable
Juin 2025	Transfert du projet à la Linux Foundation
Avril 2025	Lancement par Google avec une cinquantaine de partenaires

Selon l'annonce AAIF, le protocole est soutenu par plus de 150 organisations. Google Cloud, AWS Bedrock AgentCore Runtime et Microsoft Azure AI Foundry l'intègrent nativement, tout comme ServiceNow, Salesforce, Atlassian et SAP côté SaaS, et LangGraph, CrewAI, Pydantic AI, AG2 et IBM BeeAI côté frameworks.

Ce qu'il faut retenir de la v1.0

La v1.0 fait du fichier `a2a.proto` la **seule source normative** : JSON-RPC, gRPC et REST en dérivent, et les SDK doivent être régénérés depuis ce fichier. Elle introduit aussi la signature des Agent Cards, le multi-tenant natif, la négociation de version par en-tête `A2A-Version`, la pagination par curseur et un modèle d'erreurs unifié.

2. Concepts fondamentaux

A2A repose sur six objets : un client envoie des **Messages** (faits de **Parts**) à un agent distant décrit par son **Agent Card** ; l'agent répond soit par un Message direct, soit par une **Task** qui produit des **Artifacts**, le tout regroupé dans un **contexte**.

Les acteurs

- **Utilisateur** : humain ou système à l'origine de la demande.

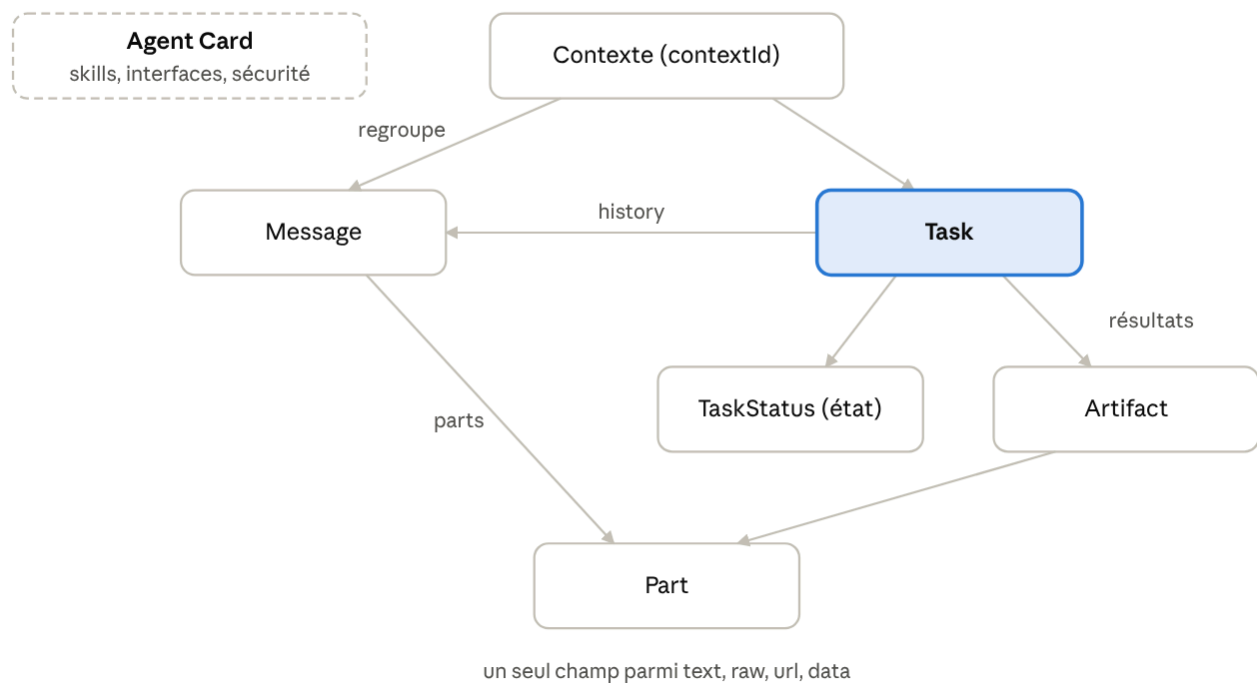
- **Client A2A (agent client)** : application ou agent qui initie les requêtes pour le compte de l'utilisateur. Un orchestrateur est typiquement un client A2A.
- **Serveur A2A (agent distant)** : agent qui expose un point d'accès conforme, traite les tâches et reste une boîte noire pour le client.

Un même agent peut être serveur pour ses appelants et client pour ses sous-traitants : c'est ce qui permet les chaînes de délégation.

Les objets du modèle

Objet	Rôle	Champs clés (v1.0)
Agent Card	Carte d'identité publique de l'agent	name, description, supportedInterfaces[], capabilities, skills[], securitySchemes, signatures
Message	Un tour de parole, client ou agent	messageId (créé par l'émetteur), role (ROLE_USER / ROLE_AGENT), parts[], contextId, taskId, referenceTaskIds[], extensions[]
Part	Plus petite unité de contenu	exactement un de text, raw (octets, base64 en JSON), url, data (JSON); plus mediaType, filename, metadata
Task	Unité de travail avec état et cycle de vie	id (généré par le serveur), contextId, status, artifacts[], history[], metadata
Artifact	Résultat produit par une tâche	artifactId (unique dans la tâche), name, description, parts[] (au moins une)
Contexte	Regroupement logique de tâches et messages	contextId, opaque pour le client

Comment s'imbriquent les objets du protocole



modèle de données · 7 objets

Un contexte regroupe messages et tâches ; une tâche porte son état, son historique de messages et ses artifacts ; messages et artifacts sont faits des mêmes Parts. L'Agent Card vit à part : elle décrit l'agent, pas les échanges.

Message ou Artifact : la règle d'or

Les Messages servent à **communiquer** : lancer une tâche, demander une précision, informer de l'avancement, fournir une entrée complémentaire. Les Artifacts servent à **livrer** le résultat. La spécification demande explicitement de ne pas livrer les résultats d'une tâche via des Messages.

Conséquence pratique : un message n'est pas un canal fiable. L'agent choisit quels messages il persiste dans l'historique, et un client en streaming qui se reconnecte peut en avoir perdu. Toute information critique doit donc transiter par un Artifact ou par l'état de la tâche.

Message direct ou Task

Face à un `SendMessage`, l'agent distant choisit :

- un **Message direct** pour une interaction simple et immédiate (question factuelle, salutation, refus rapide), sans suivi d'état ;
- une **Task** dès qu'il y a traitement, durée, résultat à livrer ou interaction multi-tour.

Le client doit gérer les deux cas. En pratique, la plupart des agents métier renvoient systématiquement une Task.

Identifiants et multi-tour

Identifiant	Qui le génère	Règles essentielles
<code>messageId</code>	L'émetteur du message	Sert à détecter les doublons (idempotence optionnelle de <code>SendMessage</code>)
<code>taskId</code>	Toujours le serveur	Un client ne peut pas créer de tâche avec son propre ID ; un <code>taskId</code> inconnu renvoie <code>TaskNotFoundError</code>
<code>contextId</code>	Le serveur en général	Le serveur peut accepter un <code>contextId</code> client, sinon il doit rejeter la requête ; un couple <code>contextId</code> / <code>taskId</code> incohérent est rejeté

Trois façons de poursuivre une conversation : réutiliser le `taskId` pour affiner une tâche en cours, réutiliser le `contextId` sans `taskId` pour lancer une nouvelle tâche dans la même conversation, ou citer d'anciennes tâches via `referenceTaskIds`. Une tâche en état terminal n'accepte plus de message : on repart d'une nouvelle tâche dans le même contexte.

Exemple de message v1.0 (JSON)

```
{
  "messageId": "b7f3c2e0-1a2b-4c3d-9e8f-001122334455",
  "role": "ROLE_USER",
  "parts": [
    { "text": "Résume ce contrat en 10 points", "mediaType": "text/plain" },
    { "url": "https://files.example.com/contrat.pdf", "filename":
"contrat.pdf", "mediaType": "application/pdf" },
    { "data": { "langue": "fr", "niveau": "direction" }, "mediaType":
"application/json" }
  ]
}
```

Il n'y a pas de champ discriminant : le type d'une Part se déduit du champ présent (`text`, `raw`, `url` ou `data`).

3. Architecture en couches et bindings

La spécification v1.0 est organisée en trois couches indépendantes : un modèle de données canonique (Protocol Buffers), des opérations abstraites, et des bindings qui projettent ces opérations sur JSON-RPC, gRPC ou HTTP+JSON/REST. Ajouter un transport ne touche jamais au modèle.

```

flowchart TB
    subgraph L1[Couche 1 : modèle de données, a2a.proto]
        T[Task] --- M[Message] --- C[AgentCard] --- P[Part] --- A[Artifact] --- E[Extension]
    end
    subgraph L2[Couche 2 : opérations abstraites]
        O1[SendMessage] --- O2[SendStreamingMessage] --- O3[GetTask] --- O4[ListTasks] --- O5[CancelTask] --- O6[SubscribeToTask]
    end
    subgraph L3[Couche 3 : bindings]
        B1[JSON-RPC 2.0] --- B2[gRPC] --- B3[HTTP+JSON/REST] --- B4[Bindings personnalisés]
    end
    L1 --> L2 --> L3
  
```

Les 11 opérations du protocole

Opération	Rôle	Capacité requise
SendMessage	Envoyer un message, recevoir Task ou Message	Aucune
SendStreamingMessage	Idem avec flux d'événements temps réel	streaming
GetTask	Lire l'état d'une tâche (polling)	Aucune
ListTasks	Lister les tâches visibles, filtrées et paginées par curseur	Aucune
CancelTask	Demander l'annulation, sans garantie de succès	Aucune
SubscribeToTask	Se (re)brancher sur le flux d'une tâche non terminée	streaming

Opération	Rôle	Capacité requise
Create / Get / List / DeleteTaskPushNotificationConfig	Gérer les webhooks d'une tâche	pushNotifications
GetExtendedAgentCard	Obtenir la carte détaillée après authentification	extendedAgentCard

Si le client appelle une opération dont la capacité n'est pas déclarée, le serveur doit répondre `UnsupportedOperationError` ou `PushNotificationNotSupportedError`. D'où la règle : lire l'Agent Card avant d'appeler.

Correspondance entre bindings

Opération	JSON-RPC (méthode)	REST (HTTP)	gRPC
SendMessage	SendMessage	POST /message:send	SendMessage
SendStreamingMessage	SendStreamingMessage (réponse SSE)	POST /message:stream (SSE)	stream serveur
GetTask	GetTask	GET /tasks/{id}	GetTask
ListTasks	ListTasks	GET /tasks	ListTasks
CancelTask	CancelTask	POST /tasks/{id}:cancel	CancelTask
SubscribeToTask	SubscribeToTask (SSE)	GET /tasks/{id}:subscribe (SSE)	stream serveur
Push config	CreateTaskPushNotificationConfig, etc.	/tasks/{taskId}/pushNotificationConfigs/{id}	idem
Carte étendue	GetExtendedAgentCard	GET /extendedAgentCard	GetExtendedAgentCard

Toutes les routes REST acceptent un préfixe `/tenant/`. Le type de média enregistré est `application/a2a+json`.

Choisir son binding

Binding	Points forts	Points faibles	Cas d'usage
JSON-RPC 2.0	Le plus répandu, un seul point d'entrée, SSE simple	Moins lisible pour les passerelles API, un seul endpoint à router	Défaut de la plupart des SDK et frameworks

Binding	Points forts	Points faibles	Cas d'usage
HTTP+JSON/REST	Routes lisibles, cache et WAF faciles, outillage API management	Plus de routes à exposer	Exposition via API gateway, équipes habituées à REST
gRPC	Performance, typage fort, streaming natif	Moins accessible depuis le navigateur, proxies spécifiques	Trafic interne à fort volume, maillage de services

Un agent déclare ses interfaces dans `supportedInterfaces[]`, par ordre de préférence. Chaque entrée précise `url`, `protocolBinding` et `protocolVersion`. Un même agent peut donc exposer JSON-RPC, REST et gRPC en parallèle, chacun à sa propre URL.

Versionnement et paramètres de service

- Le client **doit** envoyer l'en-tête `A2A-Version` (valeur `1.0`) sur chaque requête ; il peut aussi le passer en paramètre de requête.
- Seuls majeur et mineur comptent : les numéros de patch ne se négocient pas.
- Version non supportée : `VersionNotSupportedError`.
- L'en-tête `A2A-Extensions` liste les URI d'extensions que le client active pour la requête.
- Chaque interface de l'Agent Card porte sa propre `protocolVersion` : le client choisit l'interface compatible au lieu de deviner.

Modèle d'erreurs unifié

Toutes les erreurs portent un code, un message et des détails typés au format `google.rpc.Status`. Les erreurs A2A ajoutent un `google.rpc.ErrorInfo` avec `domain: "a2a-protocol.org"` et un `reason` en majuscules (ex. `TASK_NOT_FOUND`).

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "error": {
    "code": -32001,
    "message": "Task not found",
    "data": [{
      "@type": "type.googleapis.com/google.rpc.ErrorInfo",
      "reason": "TASK_NOT_FOUND",
      "domain": "a2a-protocol.org",
      "metadata": { "taskId": "123" }
    }]
  }
}
```

Erreurs spécifiques : `TaskNotFoundError`, `TaskNotCancelableError`, `PushNotificationNotSupportedError`, `UnsupportedOperationError`, `ContentTypeNotSupportedError`, `InvalidAgentResponseError`, `ExtendedAgentCardNotConfiguredError`, `ExtensionSupportRequiredError`, `VersionNotSupportedError`. Un serveur ne doit pas distinguer « n'existe pas » de « non autorisé », pour ne rien divulguer.

4. Cycle de vie des tâches, streaming et notifications push

Une Task traverse une machine à états à 8 valeurs utiles, dont 4 terminales et 2 « interrompues » ; le client suit sa progression par polling, par streaming (SSE ou gRPC) ou par webhooks.

Les états

```
stateDiagram-v2
    [*] --> SUBMITTED
    SUBMITTED --> WORKING
    SUBMITTED --> REJECTED
    WORKING --> INPUT_REQUIRED
    WORKING --> AUTH_REQUIRED
    INPUT_REQUIRED --> WORKING : nouveau message (même taskId)
    AUTH_REQUIRED --> WORKING : credentials fournis
    WORKING --> COMPLETED
    WORKING --> FAILED
    WORKING --> CANCELED : CancelTask
    WORKING --> REJECTED
    COMPLETED --> [*]
    FAILED --> [*]
    CANCELED --> [*]
    REJECTED --> [*]
```

État (v1.0)	Catégorie	Signification
<code>TASK_STATE_SUBMITTED</code>	Actif	Reçue et acquittée
<code>TASK_STATE_WORKING</code>	Actif	En cours de traitement
<code>TASK_STATE_INPUT_REQUIRED</code>	Interrompu	L'agent attend une précision du client
<code>TASK_STATE_AUTH_REQUIRED</code>	Interrompu	Une authentification supplémentaire est nécessaire
<code>TASK_STATE_COMPLETED</code>	Terminal	Succès
<code>TASK_STATE_FAILED</code>	Terminal	Échec
<code>TASK_STATE_CANCELED</code>	Terminal	Annulée avant la fin
<code>TASK_STATE_REJECTED</code>	Terminal	L'agent refuse de la traiter, au départ ou en cours

`TASK_STATE_UNSPECIFIED` existe pour la conformité Protobuf mais ne doit pas être émis volontairement. Une tâche terminale est immuable : tout nouveau message renvoie `UnsupportedOperationError`.

Mode bloquant ou non bloquant

Par défaut, `SendMessage` est **bloquant** : il attend un état terminal ou interrompu et renvoie la tâche complète avec ses artifacts. Avec `configuration.returnImmediately: true`, il renvoie la tâche dès sa création (souvent en `WORKING`) ; au client ensuite de suivre la tâche. Ce paramètre ne s'applique ni aux réponses de type `Message` ni au streaming.

Les trois mécanismes de suivi

Mécanisme	Opérations	Latence	Le client doit	Idéal pour
Polling	<code>GetTask</code> en boucle	Élevée	Rien de spécial	Intégrations simples, pare-feu restrictifs
Streaming	<code>SendStreamingMessage</code> , <code>SubscribeToTask</code>	Faible	Maintenir une connexion ouverte	UI interactives, suivi temps réel
Push (webhook)	<code>CreateTaskPushNotificationConfig</code> ou config dans <code>SendMessage</code>	Moyenne	Exposer un endpoint HTTPS joignable	Tâches longues, serveur à serveur, architectures événementielles

Le streaming en détail

Un flux commence **toujours** par un objet complet : soit un `Message` unique (le flux se ferme aussitôt), soit la `Task`. Suivent des objets `StreamResponse` contenant exactement un champ parmi `task`, `message`, `statusUpdate`, `artifactUpdate`. Le flux se ferme quand la tâche atteint un état terminal.

```
event 1 { "task": { "id": "t-42", "status": { "state":
"TASK_STATE_SUBMITTED" } } }
event 2 { "statusUpdate": { "taskId": "t-42", "status": { "state":
"TASK_STATE_WORKING" } } }
event 3 { "artifactUpdate": { "taskId": "t-42", "artifact": { "artifactId":
"a1", "parts": [{"text": "Point 1..."}] }, "append": false } }
event 4 { "artifactUpdate": { "taskId": "t-42", "artifact": { "artifactId":
"a1", "parts": [{"text": "Point 2..."}] }, "append": true, "lastChunk": true }
}
event 5 { "statusUpdate": { "taskId": "t-42", "status": { "state":
"TASK_STATE_COMPLETED" } } }
(fermeture du flux)
```

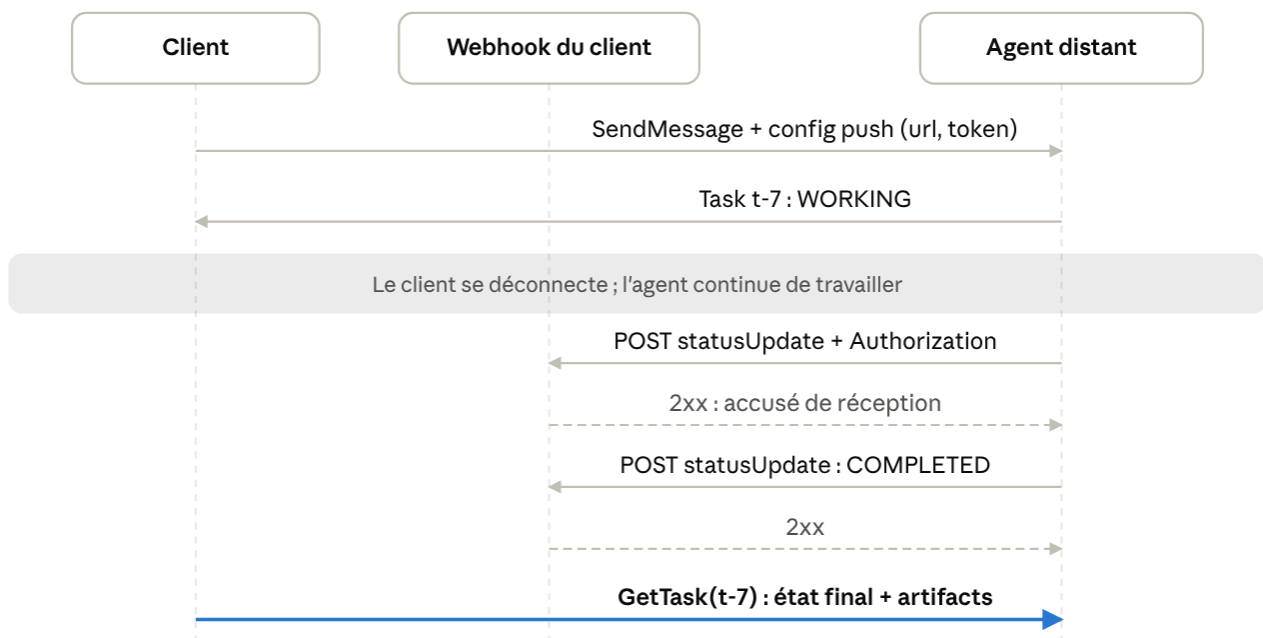
Points à retenir :

- **Ordre garanti** : les événements ne sont jamais réordonnés.
- **Découpage des artifacts** : `append: true` concatène au fragment précédent de même `artifactId`, `lastChunk: true` marque le dernier morceau.
- **Flux multiples** : plusieurs clients peuvent suivre la même tâche ; chacun reçoit les mêmes événements dans le même ordre, et fermer un flux n'affecte ni les autres ni la tâche.
- **Reconnexion** : après une coupure, `SubscribeToTask` renvoie d'abord l'état courant de la tâche, ce qui évite le trou entre un `GetTask` et un abonnement. Les messages de statut émis pendant la coupure peuvent être perdus : d'où la règle de ne rien confier de critique aux messages.

Les notifications push en détail

Le client enregistre une URL de webhook, un `token` optionnel propre à la tâche et un bloc `authentication` (schéma IANA, par exemple `Bearer`). À chaque changement, l'agent envoie un `POST` dont le corps est un `StreamResponse`, en JSON quel que soit le binding de l'agent.

Notifications push : le client peut se déconnecter



notifications push · 3 acteurs

Le client n'a pas besoin de rester connecté : l'agent pousse chaque changement vers le webhook, qui acquitte en 2xx. Si un POST échoue, l'agent réessaie ; le `GetTask` final reste la source de vérité.

Côté serveur (agent)	Côté client (récepteur)
Livraison « au moins une fois »	Répondre en 2xx pour acquitter
Retry avec backoff exponentiel autorisé	Traiter de façon idempotente (doublons possibles)
Timeout recommandé : 10 à 30 s	Vérifier que le taskId est attendu
Peut abandonner après N échecs consécutifs	Vérifier l'authenticité de l'émetteur
La config persiste jusqu'à la fin de la tâche ou sa suppression	Relire l'état complet via <code>GetTask</code> en cas de doute

La garantie « au moins une fois » sans garantie d'ordre global côté webhook impose au récepteur de raisonner en état (dernier état connu, horodatage) plutôt qu'en flux d'événements. C'est le point où un bus d'événements (Kafka, Pub/Sub) derrière le webhook apporte de la robustesse : voir la section 9.

Multi-tour et humain dans la boucle

```
sequenceDiagram
    participant C as Client (orchestrateur)
    participant A as Agent distant
    C->>A: SendMessage("Réserve un vol Paris-Tokyo")
    A-->>C: Task t1, INPUT_REQUIRED ("Quelles dates ?")
    C->>C: demande à l'utilisateur
    C->>A: SendMessage(taskId=t1, contextId=c1, "du 3 au 10 mai")
    A-->>C: Task t1, WORKING puis COMPLETED + artifact (billet)
    C->>A: SendMessage(contextId=c1, referenceTaskIds=[t1], "Ajoute un hôtel")
    A-->>C: Task t2 (nouvelle tâche, même contexte)
```

L'état `AUTH_REQUIRED` suit la même mécanique pour l'autorisation en cours de tâche (section 6). Le client répond dans la même tâche, avec le même `taskId`.

4 bis. Résilience : crash serveur et coupure réseau

En A2A, la seule source de vérité est l'**état de la tâche stocké côté serveur** : réponse HTTP, flux SSE, webhook et messages de statut peuvent tous se perdre. Être résilient consiste donc à rendre cet état durable côté serveur et à donner au client les moyens de le retrouver après n'importe quelle panne.

Ce qui peut casser, et ce que le client voit

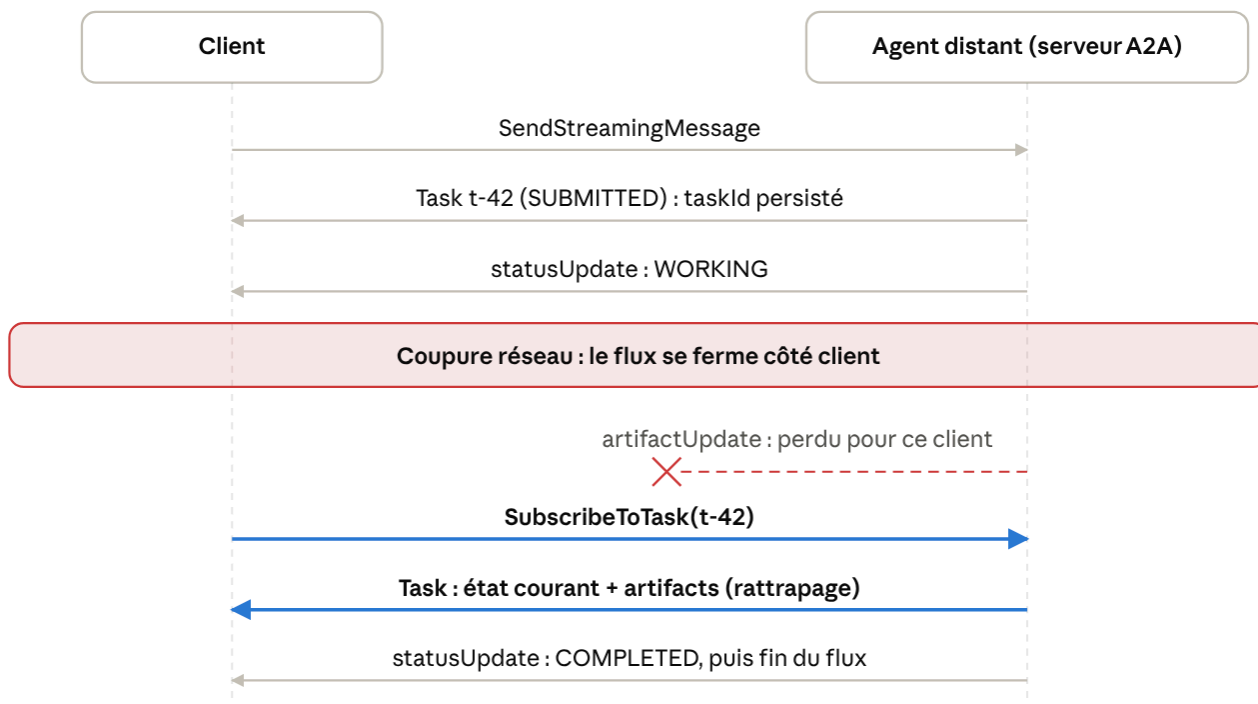
Incident	Symptôme côté client	Risque principal
Coupure réseau pendant un <code>SendMessage</code> bloquant	Timeout, aucune information sur la tâche	Relance naïve qui crée un doublon, tâche orpheline qui consomme du LLM
Coupure réseau pendant un flux	Flux interrompu	Événements et messages de statut manqués, client « aveugle »
Tâche terminée pendant la coupure	<code>SubscribeToTask</code> refusé (<code>UnsupportedOperationError</code>)	Le client croit à une panne alors que le résultat est prêt
Proxy ou load balancer qui coupe les connexions longues	Coupures régulières des appels bloquants et des flux SSE	Faux incidents, relances en rafale
Crash serveur avec store en mémoire	<code>TaskNotFoundError</code> après redémarrage	Tâche, historique et artifacts perdus
Crash serveur avec store persistant	Tâche figée en <code>TASK_STATE_WORKING</code>	Tâche « zombie » : plus aucun processus ne la fait avancer
Crash serveur au milieu d'un effet de bord	Rien de visible	Action exécutée deux fois à la reprise (e-mail, paiement, écriture)
Crash du client ou de l'orchestrateur	<code>taskId</code> oublié	Tâche orpheline, coût gaspillé, doublon à la relance
Webhook du client injoignable	Aucune notification	L'agent abandonne après N échecs : le client n'est jamais informé

Coupure réseau pendant un flux : le scénario type

Le premier événement d'un flux est toujours la Task : dès qu'il est reçu, le client connaît le `taskId` et peut se rebrancher. `SubscribeToTask` renvoie d'abord l'**état courant complet** (statut et artifacts), ce qui rattrape tout ce qui s'est passé pendant la coupure, sauf les

messages de statut transitoires.

Coupure réseau pendant un flux, puis reprise



reprise après coupure : 2 acteurs, 1 événement perdu

Seul l'événement émis pendant la coupure est manqué : l'état renvoyé par `SubscribeToTask` le rattrape, artifacts compris.

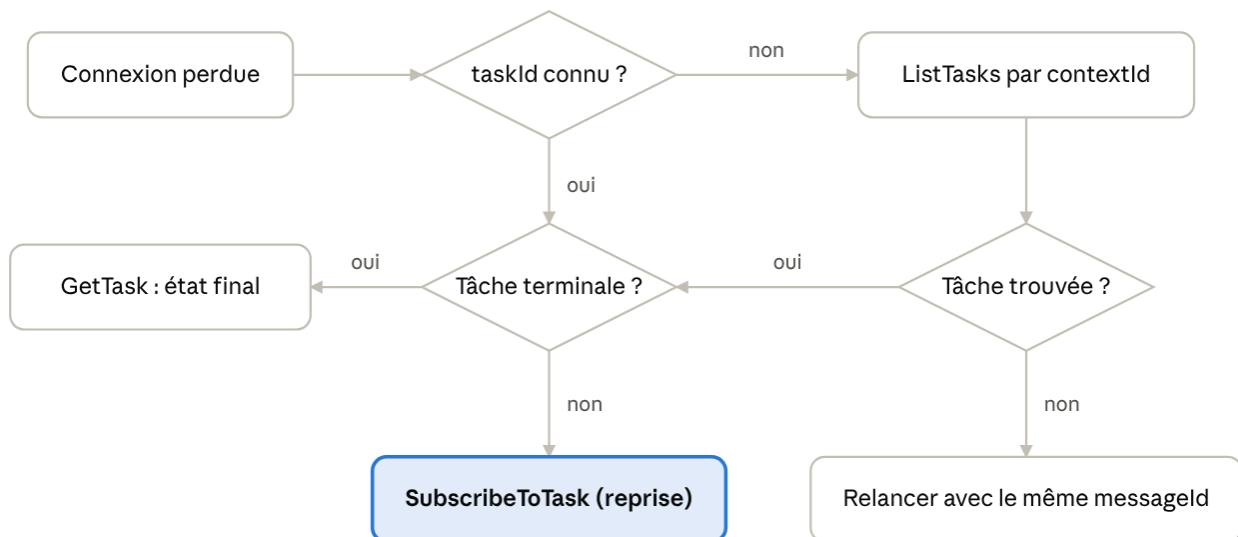
Coupure réseau : les remèdes côté client

- **Obtenir et persister le `taskId` au plus tôt** : premier événement du flux, ou `returnImmediately: true` en mode non bloquant. L'écrire en base avec le `messageId` et le `contextId` avant toute autre action.
- **Relancer avec le même `messageId`** : la spécification permet au serveur de déduplicer sur cet identifiant. Ne jamais générer un nouvel identifiant pour une relance, et vérifier dans la documentation de l'agent que la déduplication est implémentée.
- **Retrouver une tâche inconnue** : si l'agent accepte un `contextId` fourni par le client, `ListTasks` filtré sur ce contexte permet de retrouver une tâche créée juste avant la coupure, au lieu d'en relancer une.
- **Se rebrancher plutôt que relancer** : `SubscribeToTask` si la tâche est encore active, `GetTask` si elle est terminale.
- **Backoff exponentiel avec aléa, plafonné**, et disjoncteur par agent distant pour ne pas saturer un agent qui redémarre.

- **Ne rien confier de critique aux messages de statut** : seuls l'état et les artifacts font foi.
- **Préférer le non bloquant pour les tâches longues** : un `SendMessage` bloquant de plusieurs minutes finit coupé par un proxy. Aligner les délais d'inactivité des proxies et load balancers sur l'usage SSE.

L'algorithme de reprise tient en un arbre de décision.

Reprise côté client après une coupure



reprise côté client · 3 décisions, 3 issues

La relance n'intervient qu'en dernier recours, et toujours avec le même `messageId` pour permettre la déduplication côté serveur.

Crash serveur : ce qui se passe réellement

La coroutine qui exécutait la tâche meurt avec le processus. Avec un store en mémoire, la tâche disparaît. Avec un store persistant, elle survit mais reste en `WORKING` pour toujours : **le protocole ne contient aucun mécanisme qui la fera avancer**, c'est à l'implémentation de le prévoir. Les flux ouverts sont coupés, les configurations push stockées en mémoire sont perdues (la spécification exige pourtant qu'elles persistent jusqu'à la fin de la tâche), et un effet de bord à moitié fait risque d'être rejoué.

Crash serveur : l'architecture qui encaisse

Le principe est de séparer trois choses que le prototype mélange dans un seul processus : la réception des requêtes, l'exécution du travail et la diffusion des événements.

architecture résiliente · 9 composants

Une instance ou un worker peut tomber sans perdre la tâche : l'état vit dans le store, le travail dans la file, et le superviseur reprend ou clôt les tâches dont le bail a expiré.

Remède	Ce qu'il protège	Mise en œuvre
TaskStore persistant	Tâches, historique, artifacts	Base SQL (le SDK Python propose un extra SQLAlchemy pour PostgreSQL et MySQL)
Configurations push persistées	Notifications après redémarrage	Même base que les tâches
File de travaux durable	Le travail ne dépend plus de la requête HTTP	Kafka, SQS, Pub/Sub ou table de jobs ; workers sans état
Baux et battements de cœur	Détection des tâches zombies	Chaque worker renouvelle un bail sur sa tâche ; un superviseur reprend la tâche ou la passe en <code>TASK_STATE_FAILED</code> à expiration
Points de reprise	Ne pas refaire tout le travail LLM	Sauvegarde de l'état de l'agent après chaque étape (exécution durable, checkpointers du framework d'agent)
Idempotence des effets de bord	Pas de double paiement ni de double e-mail	Clé d'idempotence dérivée du <code>taskId</code> et de l'étape, pattern outbox
Bus d'événements partagé	<code>SubscribeToTask</code> servi par n'importe quelle instance	Pub/sub commun à toutes les instances ; à défaut, affinité de session
Émetteur push avec retry	Livraison malgré un webhook lent	Backoff, timeout de 10 à 30 s, abandon après N échecs
Arrêt gracieux	Déploiements sans casse	Sur SIGTERM : refuser les nouvelles tâches, finir ou rendre les baux, fermer proprement les flux

Règle d'or après un crash : une tâche qui ne peut pas être reprise passe en `TASK_STATE_FAILED` avec un message clair (par exemple « interrompue par un incident, relancez la demande »). Une tâche ne doit jamais rester indéfiniment en `WORKING`. Une tâche reprise garde son `taskId` et reste en `WORKING` : le protocole n'a pas d'état « reprise »,

Crash du client et webhook injoignable

- **Journal avant envoi** : persister `messageId` avant d'appeler, puis `taskId` et `contextId` dès réception. Au redémarrage, le client sait quelles tâches suivre.
- **Réconciliation périodique** : `ListTasks` avec `statusTimestampAfter` égal au dernier horodatage traité rattrape toutes les tâches modifiées pendant la panne, y compris celles dont le webhook a échoué. C'est le filet de sécurité des notifications push.
- **Webhook idempotent** : dédupliquer sur le couple `taskId` et horodatage du statut, et raisonner en dernier état connu plutôt qu'en suite d'événements.
- **Nettoyer** : annuler par `CancelTask` (idempotent) les tâches devenues inutiles, pour ne pas payer un travail que plus personne n'attend.

5. Découverte des agents et Agent Card

L'Agent Card est le contrat d'un agent : identité, points d'accès, capacités, compétences et exigences de sécurité, publiés en JSON, idéalement à `https://{domaine}/.well-known/agent-card.json` (RFC 8615). Un client bien conçu ne fait aucune hypothèse qu'il n'a pas lue dans la carte.

Anatomie d'une Agent Card v1.0

Champ	Obligatoire	Contenu
<code>name</code> , <code>description</code> , <code>version</code>	Oui	Identité et version de l'agent (pas du protocole)
<code>supportedInterfaces[]</code>	Oui	Liste ordonnée d'interfaces : <code>url</code> , <code>protocolBinding</code> , <code>protocolVersion</code> , <code>tenant</code> optionnel ; la première est préférée
<code>capabilities</code>	Oui	<code>streaming</code> , <code>pushNotifications</code> , <code>extendedAgentCard</code> , <code>extensions[]</code>
<code>defaultInputModes</code> , <code>defaultOutputModes</code>	Oui	Types de média acceptés et produits

Champ	Obligatoire	Contenu
<code>skills[]</code>	Oui	Compétences : <code>id</code> , <code>name</code> , <code>description</code> , <code>tags</code> , <code>examples</code> , modes d'entrée et sortie propres
<code>securitySchemes</code> , <code>securityRequirements</code>	Non	Schémas d'authentification acceptés (voir section 6)
<code>provider</code>	Non	Organisation et URL de l'éditeur
<code>signatures[]</code>	Non	Signatures JWS de la carte
<code>documentationUrl</code> , <code>iconUrl</code>	Non	Documentation et icône

Exemple commenté

```
{
  "name": "Agent de synthèse documentaire",
  "description": "Résumé des documents longs (PDF, DOCX, HTML) en français ou
en anglais.",
  "version": "2.3.0",
  "provider": { "organization": "Exemple SA", "url": "https://exemple.fr" },
  "supportedInterfaces": [
    { "url": "https://agents.exemple.fr/summarizer", "protocolBinding":
"JSONRPC", "protocolVersion": "1.0" },
    { "url": "https://agents.exemple.fr/summarizer/rest", "protocolBinding":
"HTTP+JSON", "protocolVersion": "1.0" }
  ],
  "capabilities": { "streaming": true, "pushNotifications": true,
"extendedAgentCard": true },
  "defaultInputModes": ["text/plain", "application/pdf"],
  "defaultOutputModes": ["text/plain", "application/json"],
  "skills": [{
    "id": "summarize",
    "name": "Synthèse",
    "description": "Produit une synthèse structurée d'un document.",
    "tags": ["synthèse", "résumé", "documents"],
    "examples": ["Résume ce contrat en 10 points"]
  }],
  "securitySchemes": {
    "oauth": { "oauth2SecurityScheme": { "flows": { "clientCredentials": {
      "tokenUrl": "https://auth.exemple.fr/token", "scopes": { "summarize":
"Lancer une synthèse" } } } } }
  }
}
```

```

},
"securityRequirements": [{ "schemes": { "oauth": { "list": ["summarize"] }
} }]
}

```

Les noms exacts des sous-champs de sécurité sont dérivés du `.proto` : vérifier la forme générée par votre SDK plutôt que de les écrire à la main.

Trois stratégies de découverte

Stratégie	Mécanisme	Avantages	Limites	Contexte
URI bien connue	GET <code>/.well-known/agent-card.json</code> sur le domaine de l'agent	Standard, simple, automatisable	Il faut connaître le domaine ; carte publique par défaut	Agents publics ou d'un domaine maîtrisé
Registre (catalogue)	Les agents publient leur carte ; les clients cherchent par compétence, tag, fournisseur	Gouvernance centrale, recherche par capacité, divulgation sélective	Aucune API de registre standardisée à ce jour ; service à opérer	Entreprise, places de marché
Configuration directe	URL ou carte en dur, fichier de config, variables d'environnement	Immédiat, zéro infrastructure	Rigide, reconfiguration à chaque changement	Développement, systèmes fortement couplés

découverte · 3 sources, 1 contrôle de signature

Quelle que soit la source, le client vérifie la signature de la carte, choisit dans `supportedInterfaces` un binding et une version qu'il sait parler, puis s'authentifie selon les `securitySchemes` avant le premier appel.

Le registre est le chaînon manquant du standard : chaque éditeur de plateforme propose le sien. Concevoir son registre avec une API interne stable, en attendant une normalisation, est le choix le plus prudent.

Carte publique et carte étendue

La carte publique ne doit contenir que ce qui peut être vu par tous. Si `capabilities.extendedAgentCard` vaut `true`, un client authentifié peut appeler `GetExtendedAgentCard` pour obtenir une version enrichie : compétences sensibles, quotas, points d'accès internes. Le client remplace alors sa carte en cache par la version étendue pour la durée de sa session.

La carte peut aussi être protégée directement : mTLS, filtrage réseau, OAuth sur l'endpoint well-known, ou divulgation sélective par le registre. Règle absolue : **aucun secret statique dans une carte**, uniquement des références vers des mécanismes d'obtention de credentials.

Signature des cartes

Une carte peut être signée pour prouver son origine et son intégrité, ce qui protège contre l'usurpation d'agent et la modification en transit ou dans un registre.

1. Retirer le champ `signatures` de la carte.
2. Canonicaliser le JSON selon RFC 8785 (JCS) : ordre des clés et format des nombres déterministes.
3. Signer avec JWS (RFC 7515), en signature détachée.
4. Publier la signature dans `signatures[]` ; la clé publique est obtenue via `jku` ou un magasin de clés de confiance.

Le client refait la canonicalisation, vérifie la signature et rejette la carte en cas d'échec. La signature prouve **qui** a publié la carte, pas que l'agent est **fiable** : la politique de confiance (quels émetteurs, quelles clés) reste à définir par chaque organisation.

Cache

Côté serveur : en-têtes `Cache-Control` avec `max-age` et `ETag` dérivé de `version` ou d'un hash. Côté client : respecter la sémantique HTTP, revalider par `If-None-Match`, appliquer une durée par défaut raisonnable si le serveur n'en donne pas. Une carte change rarement, mais un client qui ne la rafraîchit jamais ratera un changement d'authentification ou de version.

6. Sécurité : authentification, autorisation, confiance

A2A n'invente aucun mécanisme de sécurité : il déclare dans l'Agent Card quels standards web l'agent accepte (TLS, OAuth 2.0, OIDC, mTLS, clés d'API) et laisse l'obtention des credentials hors bande. La sécurité réelle d'un système A2A dépend donc surtout de l'implémentation et de la gouvernance, pas du protocole.

Transport

HTTPS est obligatoire en production, avec une version moderne de TLS. Le client vérifie l'identité du serveur par son certificat. Les credentials voyagent dans les en-têtes HTTP (ou les métadonnées gRPC), jamais dans le corps des messages A2A.

Schémas d'authentification déclarables

Schéma	Usage type	Remarque
API Key (en-tête, query, cookie)	Prototypes, partenaires simples	Secret statique, rotation à gérer
HTTP Auth (Bearer, Basic)	Jetons émis par un IdP existant	Bearer est le cas le plus courant
OAuth 2.0 : Authorization Code + PKCE	Agent agissant pour un utilisateur	<code>pkceRequired</code> : PKCE exigible
OAuth 2.0 : Client Credentials	Agent à agent, sans utilisateur	Le plus fréquent en serveur à serveur
OAuth 2.0 : Device Code (RFC 8628)	CLI, environnements sans navigateur	Code saisi sur un autre appareil
OpenID Connect	Identité fédérée	Découverte via l'URL OIDC
Mutual TLS	Zones de confiance internes, B2B	Identité forte des deux côtés

Les flux OAuth Implicit et Password ne font **pas partie** des flux proposés, conformément aux bonnes pratiques OAuth actuelles : utiliser Authorization Code + PKCE à la place.

Flux d'authentification

1. Le client lit `securitySchemes` et `securityRequirements` dans l'Agent Card.
2. Il obtient un jeton hors bande auprès du fournisseur d'identité indiqué.
3. Il envoie chaque requête avec l'en-tête approprié (`Authorization: Bearer ...`).
4. Le serveur authentifie chaque requête, puis décide de l'autorisation selon l'identité, les scopes et la compétence visée.
5. Échec : 401 / `UNAUTHENTICATED` avec indication du schéma attendu, ou 403 / `PERMISSION_DENIED` .

authentification · 3 acteurs, 1 interruption AUTH_REQUIRED

Les jetons s'obtiennent toujours hors bande, auprès du fournisseur d'identité désigné par la carte. Quand un droit manque en cours de route, la tâche s'interrompt en `AUTH_REQUIRED` et reprend avec le même `taskId` une fois le nouveau jeton obtenu.

Autorisation en cours de tâche

Une tâche peut découvrir en route qu'il lui faut un droit supplémentaire (accéder à l'agenda de l'utilisateur, valider un paiement). Elle passe alors en `TASK_STATE_AUTH_REQUIRED` avec un message décrivant le besoin. Le client obtient le consentement ou le jeton hors bande, puis reprend la tâche avec le même `taskId`. Ce mécanisme permet un consentement « juste à temps » au lieu d'une sur-attribution de droits au départ.

Obligations côté serveur

- **Cloisonnement des données** : `GetTask`, `ListTasks` et `SubscribeToTask` ne renvoient que les tâches visibles par l'appelant. Une tâche d'un autre utilisateur doit être indiscernable d'une tâche inexistante.
- **Validation systématique** des entrées, types de média et tailles.
- **Carte étendue** réservée aux clients authentifiés et autorisés.
- **Journalisation** des erreurs système et traçabilité des appels.

Menaces spécifiques et contre-mesures

Menace	Description	Contre-mesures
SSRF via webhook	Le client fournit une URL de push pointant vers le réseau interne de l'agent	Liste blanche de domaines, résolution DNS contrôlée, interdiction des IP privées, vérification de possession de l'URL
Webhooks falsifiés	Un tiers envoie de fausses notifications au client	Authentifier l'émetteur (jeton de la config, signature, mTLS), relire l'état par <code>GetTask</code>
Usurpation d'agent	Fausse Agent Card, typosquatting de domaine, registre compromis	Cartes signées (JWS + JCS), registre gouverné, liste de confiance des émetteurs

Menace	Description	Contre-mesures
Injection de prompt inter-agents	Un agent distant renvoie un contenu qui manipule le LLM de l'agent client	Traiter toute sortie d'agent comme donnée non fiable, séparer instructions et données, filtrage, garde-fous sur les actions
« Confused deputy » et escalade	Un agent intermédiaire réutilise un jeton trop large pour agir en aval	Jetons à portée réduite par saut (token exchange RFC 8693), audience restreinte, durée courte
Exfiltration par délégation	Données sensibles propagées le long d'une chaîne d'agents	Classification des données, minimisation dans les Parts, contrôle des destinataires
Déni de service	Tâches longues ou flux ouverts en masse	Quotas par client, limites de tâches concurrentes, timeouts, <code>Retry-After</code>

Limite importante

Le protocole transporte l'identité du **client**, pas nécessairement celle de l'utilisateur final à travers plusieurs sauts. La délégation de bout en bout (qui agit pour qui, avec quels droits réduits) doit être construite avec les mécanismes OAuth standards : la spécification ne l'impose pas. C'est l'un des sujets les plus ouverts pour les déploiements en entreprise.

7. A2A face à MCP et aux autres protocoles

A2A et MCP sont complémentaires, et désormais gouvernés sous le même toit (AAIF) : MCP est l'axe **vertical** qui relie un agent à ses outils et données, A2A l'axe **horizontal** qui relie des agents entre eux. Un agent typique est client MCP pour ses outils et serveur A2A pour ses pairs.

Comparaison détaillée

Critère	MCP (Model Context Protocol)	A2A (Agent2Agent)
Relation	Agent vers outil, ressource, prompt	Agent vers agent, entre pairs

Critère	MCP (Model Context Protocol)	A2A (Agent2Agent)
Nature de l'appelé	Fonction déterministe et transparente, schéma d'entrée précis	Système autonome, opaque, qui raisonne
Unité d'échange	Appel d'outil avec paramètres JSON Schema	Message multimodal, Task avec cycle de vie
Durée typique	Secondes	Secondes à jours
État	Peu ou pas d'état métier exposé	Tâche avec états, historique, artifacts
Dialogue	Requête / réponse	Multi-tour, demande de précision, humain dans la boucle
Découverte	Liste des outils à l'initialisation de la session	Agent Card publique, registres
Contrat d'entrée	Schéma strict par outil	Description en langage naturel des compétences (pas de schéma formel par skill)
Qui contrôle	Le LLM appelant décide et interprète	L'agent distant décide de sa méthode

Règle de décision

- La capacité est une **fonction** bien définie (lire une base, appeler une API, lancer un calcul) : l'exposer en **MCP**.
- La capacité exige du **jugement**, de la négociation, plusieurs étapes, ou appartient à une autre équipe ou un autre éditeur qui veut garder la main sur sa logique : l'exposer en **A2A**.
- Zone grise : un agent peut être enveloppé en outil MCP (appel simple, résultat unique) ; on perd alors le suivi de tâche, le multi-tour et le streaming d'artifacts.

Architecture combinée

```

flowchart LR
    U[Utilisateur] --> O[Agent orchestrateur]
    O -- A2A --> R[Agent recherche]
    O -- A2A --> F[Agent finance, autre éditeur]
    R -- MCP --> W["(Recherche web)"]
    R -- MCP --> D["(Base documentaire)"]
    F -- MCP --> ERP["(ERP)"]
    O -- MCP --> CAL["(Agenda)"]
  
```

Chaque agent garde ses outils MCP privés ; seules ses compétences sont visibles via A2A. C'est ce qui permet de faire collaborer des agents d'éditeurs différents sans exposer leur outillage.

Le reste du paysage

Protocole	Objet	Relation avec A2A
MCP	Agent vers outils et contexte	Complémentaire, même fondation (AAIF)
AG-UI	Agent vers interface utilisateur (événements vers le front)	Complémentaire : AG-UI côté humain, A2A entre agents
ACP (IBM, BeeAI)	Communication entre agents, style REST	A rejoint A2A sous la Linux Foundation en 2025 ; BeeAI supporte A2A
ANP (Agent Network Protocol)	Réseau d'agents décentralisé, identités DID	Alternative orientée web ouvert et décentralisé, adoption plus faible
Appels REST ou gRPC sur mesure	Intégration point à point	Ce qu'A2A remplace : coût $N \times N$, pas de sémantique de tâche
Frameworks multi-agents (LangGraph, CrewAI, AutoGen / AG2)	Orchestration dans un même runtime	Internes à une application ; A2A sert à franchir la frontière du framework

Le bon réflexe : les frameworks pour orchestrer **à l'intérieur** d'une application, A2A dès qu'on franchit une frontière d'équipe, de déploiement, de technologie ou d'organisation.

8. Implémentation pas à pas avec le SDK Python

Avec le SDK Python officiel (`a2a-sdk` , ligne 1.x, compatible spécification 1.0), un agent A2A se résume à trois briques : un **AgentExecutor** qui contient la logique, un **DefaultRequestHandler** qui gère le protocole et les tâches, et des **routes** (Agent Card + JSON-RPC ou REST) montées dans une application ASGI. On construit ici un agent de synthèse qui demande une précision si la longueur n'est pas indiquée.

Les chemins d'import ci-dessous suivent le tutoriel officiel 1.x : en cas d'erreur d'import, vérifier la référence API de la version installée.

Vue d'ensemble : du client à l'exécuteur

Avant d'écrire du code, voici ce que fait le SDK quand l'agent A délègue une tâche à l'agent B. Vous n'écrivez que les deux agents et l' `AgentExecutor` ; tout le reste est fourni par le SDK.

parcours d'un `SendMessage` · côté client et côté serveur

Lecture du schéma, dans l'ordre d'un appel :

1. **Agent A** (avec ses propres outils MCP, framework et modèle) décide de déléguer. Son **A2A Client** lit l'Agent Card de B pour connaître l'URL, le binding et les capacités.
2. Le client envoie une requête JSON-RPC `SendMessage` à l'**A2A Server** de B, qui la transmet au **Request Handler**.
3. Le Request Handler crée (**setup**) une **ActiveTask**, suivie par l'**ActiveTaskRegistry** qui connaît toutes les tâches en cours de l'instance, puis s'y abonne (**subscribe**).
4. L'**AgentExecutor** lance l'**Agent B**, qui publie (**enqueue**) ses événements dans l'**EventQueue** : statut `WORKING` , artifact, statut `COMPLETED` .
5. L'**ActiveTask** vide (**drain**) la file, met à jour l'état de la tâche et relaie les événements vers le Request Handler, qui répond au client par une `Task` ou un `Message` (ou par un flux SSE en streaming).

Les noms des classes internes (`ActiveTask`, registre) peuvent varier selon la version du SDK ; le principe reste : l'exécuteur produit des événements, le SDK les transforme en état de tâche et en réponses du protocole. C'est aussi pour cela que le registre et la file sont en mémoire par défaut, et qu'il faut les remplacer par des composants partagés en production (section 4 bis).

Étape 0 : environnement

```
python -m venv .venv && source .venv/bin/activate
pip install a2a-sdk uvicorn starlette httpx
# extras optionnels selon les besoins : grpc, sqlalchemy (TaskStore
persistant), opentelemetry
python -c "import a2a; print('SDK OK')"
```

Python 3.10 minimum. Les SDK officiels existent aussi en Go, Java, JavaScript, .NET et Rust.

Étape 1 : la logique métier, indépendante du protocole

```
# summarizer.py
class Summarizer:
    """Logique métier pure : aucun import A2A ici."""

    async def summarize(self, text: str, points: int):
        # Remplacer par l'appel LLM de votre choix (streaming de jetons)
        for i in range(1, points + 1):
            yield f"{i}. Point clé numéro {i} du document.\n"
```

Séparer la logique du protocole permet de tester l'une sans l'autre et de changer de framework d'agent sans toucher à la couche A2A.

Étape 2 : l'AgentExecutor

L'exécuteur reçoit un `RequestContext` (message entrant, tâche courante éventuelle) et publie des événements dans une `EventQueue` : la Task, des mises à jour de statut, des artifacts. Le `TaskUpdater` simplifie ces publications.

```
# agent_executor.py
import re
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.tasks import TaskUpdater
from a2a.helpers import (
    new_task_from_user_message, new_text_message, new_text_part,
    get_message_text,
)
from a2a.types import TaskState
from summarizer import Summarizer

class SummarizerExecutor(AgentExecutor):
```

```

def __init__(self) -> None:
    self.agent = Summarizer()

    async def execute(self, context: RequestContext, event_queue: EventQueue)
-> None:
    # 1. Récupérer ou créer la tâche
    task = context.current_task
    if task is None:
        task = new_task_from_user_message(context.message)
        await event_queue.enqueue_event(task)
        updater = TaskUpdater(event_queue=event_queue, task_id=task.id,
context_id=task.context_id)

        text = get_message_text(context.message) or ""

    # 2. Multi-tour : demander une précision si le nombre de points
manque
    match = re.search(r"(\d+)\s*points?", text)
    if match is None and context.current_task is None:
        await updater.update_status(
            state=TaskState.TASK_STATE_INPUT_REQUIRED,
            message=new_text_message("En combien de points voulez-vous la
synthèse ?"),
        )
        return # la tâche reste ouverte ; le client répondra avec le
même taskId
    points = int(match.group(1)) if match else 5

    # 3. Travail en cours
    await updater.update_status(
        state=TaskState.TASK_STATE_WORKING,
        message=new_text_message("Synthèse en cours..."),
    )

    # 4. Livrer le résultat comme artifact (ici en un seul bloc)
    chunks = [c async for c in self.agent.summarize(text, points)]
    await updater.add_artifact(
        parts=[new_text_part(text="".join(chunks),
media_type="text/plain")],
        name="synthese",
    )

    # 5. Clore la tâche
    await updater.update_status(state=TaskState.TASK_STATE_COMPLETED)

    async def cancel(self, context: RequestContext, event_queue: EventQueue)

```

```
-> None:
    task = context.current_task
    updater = TaskUpdater(event_queue=event_queue, task_id=task.id,
context_id=task.context_id)
    # Interrompre ici le travail en cours (annuler la coroutine LLM,
etc.)
    await updater.update_status(state=TaskState.TASK_STATE_CANCELED)
```

Points d'attention : le résultat part dans un **artifact**, pas dans un message ; l'état `INPUT_REQUIRED` rend la main sans clore la tâche ; en cas d'exception, publier `TASK_STATE_FAILED` avec un message exploitable plutôt que laisser la tâche en `WORKING`. Pour du streaming fin, publier plusieurs mises à jour d'artifact avec le même identifiant, le mode ajout et le marqueur de dernier morceau (paramètres exacts de `add_artifact` à vérifier dans la référence API).

Étape 3 : Agent Card et serveur

```
# __main__.py
import uvicorn
from starlette.applications import Starlette
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes,
create_jsonrpc_routes, create_rest_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface,
AgentSkill
from agent_executor import SummarizerExecutor

BASE = "http://127.0.0.1:9999"

skill = AgentSkill(
    id="summarize",
    name="Synthèse de document",
    description="Résume un texte en N points. Demande N s'il n'est pas
précisé.",
    tags=["synthèse", "résumé"],
    examples=["Résume ce texte en 5 points : ..."],
    input_modes=["text/plain"],
    output_modes=["text/plain"],
)

card = AgentCard(
    name="Agent de synthèse",
    description="Produit des synthèses structurées de documents.",
    version="1.0.0",
```

```

default_input_modes=["text/plain"],
default_output_modes=["text/plain"],
capabilities=AgentCapabilities(streaming=True, push_notifications=False),
supported_interfaces=[
    AgentInterface(protocol_binding="JSONRPC", url=f"{BASE}/",
protocol_version="1.0"),
    AgentInterface(protocol_binding="HTTP+JSON", url=f"{BASE}/rest",
protocol_version="1.0"),
],
skills=[skill],
)

handler = DefaultRequestHandler(
    agent_executor=SummarizerExecutor(),
    task_store=InMemoryTaskStore(),    # à remplacer par un store persistant
    en production
    agent_card=card,                  # le handler s'en sert pour valider les
    capacités
)

routes = []
routes += create_agent_card_routes(card)    # /.well-known/agent-
card.json
routes += create_jsonrpc_routes(handler, "/")    # binding JSON-RPC
routes += create_rest_routes(handler, "/rest")    # binding REST (signature à
vérifier)
app = Starlette(routes=routes)

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=9999)

```

Le handler gère seul le cycle de vie, la persistance via le `TaskStore`, le streaming SSE, `GetTask`, `ListTasks`, `CancelTask` et `SubscribeToTask`. Monter les routes dans Starlette ou FastAPI permet d'ajouter authentification, journalisation et middlewares standards.

Étape 4 : tester à la main

```
# Découverte
curl -s http://127.0.0.1:9999/.well-known/agent-card.json | jq

# SendMessage en JSON-RPC (v1.0)
curl -s http://127.0.0.1:9999/ \
  -H 'Content-Type: application/json' -H 'A2A-Version: 1.0' \
  -d '{
    "jsonrpc": "2.0", "id": 1, "method": "SendMessage",
    "params": { "message": {
      "messageId": "m-001", "role": "ROLE_USER",
      "parts": [{ "text": "Résume ce texte : ..." }]}
    } }' | jq

# Réponse attendue : une Task en TASK_STATE_INPUT_REQUIRED avec la question
de l'agent
```

Étape 5 : le client Python, y compris le multi-tour

```
# client.py
import asyncio, httpx
from a2a.client import A2ACardResolver, ClientConfig, create_client
from a2a.helpers import new_text_message
from a2a.types import Role, SendMessageRequest, TaskState

async def main():
    async with httpx.AsyncClient() as http:
        card = await A2ACardResolver(httpx_client=http,
        base_url="http://127.0.0.1:9999").get_agent_card()

        client = await create_client(agent=card,
        client_config=ClientConfig(streaming=True))

        # Tour 1 : sans nombre de points
        first = SendMessageRequest(message=new_text_message("Résume ce texte :
...", role=Role.ROLE_USER))
        task = None
        async for event in client.send_message(first):
            print(event)
            task = getattr(event, "task", None) or task    # conserver la tâche
initiale

        # Tour 2 : répondre dans la même tâche
        reply = new_text_message("5 points", role=Role.ROLE_USER)
        reply.task_id, reply.context_id = task.id, task.context_id
```

```

    async for event in
client.send_message(SendMessageRequest(message=reply)):
    print(event)    # WORKING, artifact, puis COMPLETED

    await client.close()

asyncio.run(main())

```

Le client choisit l'interface et le binding à partir de la carte, et envoie l'en-tête de version. Il faut toujours gérer les deux formes de réponse (Task ou Message) et tester l'état avant d'exploiter les artifacts.

Étape 6 : passer du prototype au service

Sujet	Prototype	Production
TaskStore	InMemoryTaskStore	Store persistant (PostgreSQL, MySQL via l'extra SQLAlchemy) pour survivre aux redémarrages et scaler horizontalement
Files d'événements	En mémoire, un seul process	Diffusion partagée entre instances si plusieurs réplicas servent les mêmes flux
Authentification	Aucune	Middleware OAuth / OIDC, schémas déclarés dans la carte, carte étendue
Push	Désactivé	Config push persistée, envoi avec retry, protection SSRF
Observabilité	print	OpenTelemetry (extra du SDK), corrélation par taskId et contextId
Conformité	Tests manuels	A2A Inspector et kit de tests de conformité (TCK) en CI

Tester et valider

- **A2A Inspector** : outil web pour charger une Agent Card, envoyer des messages et inspecter les échanges.
- **A2A TCK** : suite de tests de conformité à brancher dans la CI.
- **Tests unitaires** de l'exécuteur avec une EventQueue locale : vérifier la séquence d'états (SUBMITTED, WORKING, COMPLETED) et le contenu des artifacts.
- **Tests de contrat** entre agents : figer des échanges de référence et les rejouer à chaque version.

9. Patterns d'architecture et bonnes pratiques de production

A2A standardise la conversation entre deux agents, pas l'architecture d'un système multi-agents : routage, registre, résilience, observabilité et gouvernance restent à concevoir. Cinq patterns couvrent l'essentiel des déploiements.

Les patterns

Pattern	Principe	Forces	Risques	Quand l'utiliser
Orchestrateur central	Un agent décompose la demande et délègue à des agents spécialisés via A2A	Contrôle, traçabilité, point unique de politique	Goulot d'étranglement, orchestrateur « omniscient » difficile à maintenir	Assistants métier, parcours client
Délégation hiérarchique	Les agents délégués deviennent à leur tour clients d'autres agents	Découpage naturel par domaine	Latence cumulée, propagation d'erreurs, dérive des droits	Grands périmètres fonctionnels
Maillage pair à pair	Les agents se découvrent via un registre et s'appellent directement	Découplage, évolutivité	Boucles d'appels, gouvernance difficile, coûts imprévisibles	Écosystèmes inter-équipes matures
Passerelle A2A	Un proxy en frontal applique authentification, quotas, journalisation, routage par tenant	Politique centralisée, sécurité homogène	Composant critique à rendre hautement disponible	Toute exposition hors d'une équipe, ouverture à des partenaires
Pont événementiel	Les notifications push alimentent un bus (Kafka, Pub/Sub) qui distribue aux consommateurs	Découplage temporel, rejeu, absorption des pics	Complexité, idempotence obligatoire	Tâches longues, volumétrie élevée, SI déjà orienté événements

Zoom : A2A et bus d'événements

A2A reste en HTTP point à point ; il n'existe pas de binding Kafka standard. Deux intégrations tiennent la route :

1. **Webhook vers bus** : un récepteur léger valide et acquitte le push A2A (2xx rapide), puis publie l'événement dans un topic partitionné par `taskId`. Les consommateurs traitent de façon idempotente et relisent l'état via `GetTask` si besoin. Le partitionnement par tâche rétablit l'ordre que le webhook ne garantit pas.
2. **Bus vers A2A** : un consommateur transforme des événements métier en appels `SendMessage` avec `returnImmediately: true`, et stocke la correspondance événement / `taskId` pour le suivi.

pont A2A et bus d'événements · 2 sens

En haut, le récepteur acquitte vite le push et publie dans un topic partitionné par tâche, ce qui rétablit l'ordre ; en bas, le pont transforme des événements métier en `SendMessage` non bloquants.

Un binding personnalisé sur un bus est permis par la spécification (section « Custom Bindings »), mais il sort de l'interopérabilité garantie : à réserver à un usage interne, documenté comme extension.

Registre d'entreprise

Faute d'API standard, un registre interne doit au minimum : stocker les cartes (publiques et étendues), vérifier les signatures, indexer par compétence et tag, gérer le cycle de vie (brouillon, validé, déprécié), filtrer la visibilité par identité et tracer qui consomme quel agent. Prévoir une API stable et une synchronisation avec les well-known URI, pour pouvoir basculer vers un standard de registre quand il existera.

Multi-tenant

La v1.0 ajoute un champ `tenant` à toutes les requêtes et à chaque `AgentInterface`. Un seul déploiement peut ainsi servir plusieurs agents ou clients, avec un routage par préfixe `/ {tenant} /` en REST. Le tenant est un identifiant de routage opaque : il ne remplace pas l'authentification ni le cloisonnement des données.

Bonnes pratiques de conception d'un agent serveur

- **Des compétences étroites et bien décrites** : description, tags et exemples servent au routage par LLM ; une description floue produit des délégations erronées.

- **Artifacts structurés** : pour les échanges machine à machine, préférer des Parts `data` avec `mediaType: application/json` et documenter le schéma (dans la description, une extension ou la carte étendue), puisque le protocole n'impose pas de schéma par compétence.
- **États honnêtes** : `REJECTED` quand la demande sort du périmètre, `FAILED` avec message exploitable, jamais de tâche bloquée en `WORKING`.
- **Idempotence** : détecter les doublons par `messageId`, surtout derrière des retries.
- **Politique de rétention** des tâches et contextes documentée (durée, purge).
- **Cancel réel** : l'annulation doit interrompre le travail et les coûts LLM, pas seulement changer l'état.

Bonnes pratiques côté client et orchestrateur

- Lire la carte, vérifier capacités et version avant d'appeler ; mettre la carte en cache avec revalidation.
- **Timeouts, retries et disjoncteurs** : le protocole les laisse explicitement au client.
- **Budget de délégation** : profondeur maximale de chaîne, nombre maximal d'agents appelés, plafond de coût par requête utilisateur, pour éviter boucles et explosions de coûts.
- Traiter toute sortie d'agent tiers comme une **entrée non fiable** (injection de prompt).
- Reprendre après coupure avec `SubscribeToTask` ou `GetTask`, jamais en relançant la demande.

Observabilité et gouvernance

Axe	Pratique recommandée
Traces	Propager le contexte W3C Trace Context dans les en-têtes, un span par appel A2A, attributs taskId, contextId, compétence, agent appelé
Métriques	Latence par état, taux de FAILED / REJECTED, durée des tâches, profondeur des chaînes, coût LLM par tâche
Journaux	Corrélation par contextId de bout en bout, sans contenu sensible en clair
Audit	Qui a délégué quoi à quel agent, avec quels droits, pour quel utilisateur

Axe	Pratique recommandée
Catalogue	Propriétaire, SLA, classification des données et statut de chaque agent
Évaluation	Jeux de tests par compétence, rejoués à chaque nouvelle version d'agent

Versionner sans casser

Distinguer trois versions : celle du **protocole** (par interface, négociée par `A2A-Version`), celle de l'**agent** (`version` dans la carte) et celle des **extensions** (dans leur URI). Exposer temporairement deux interfaces lors d'une montée de version du protocole, faire évoluer les compétences par ajout plutôt que par modification, et annoncer les dépréciations dans la carte étendue et le registre.

10. Avantages, inconvénients et limites

Bilan en une phrase : A2A est devenu en 2026 le standard de fait pour faire dialoguer des agents hétérogènes, avec une spécification stable et une gouvernance neutre, mais il laisse ouverts la découverte à grande échelle, la délégation d'identité et les contrats de données, qui retombent sur les architectes.

Avantages

Avantage	Pourquoi c'est important
Interopérabilité réelle	Un agent LangGraph, CrewAI, Pydantic AI, Salesforce ou ServiceNow peut en appeler un autre sans code d'intégration spécifique : coût d'intégration en N au lieu de $N \times N$
Exécution opaque	Chaque éditeur ou équipe garde sa logique, ses prompts, ses outils et ses données ; seules les compétences sont exposées
Modèle de tâche riche	États, multi-tour, humain dans la boucle, tâches de plusieurs jours, annulation : ce qu'aucune API REST ad hoc ne fournit d'emblée
Trois modes de suivi	Polling, streaming, webhooks : adapté aussi bien à une UI qu'à une intégration serveur à serveur

Avantage	Pourquoi c'est important
Standards web éprouvés	HTTP, JSON-RPC, gRPC, SSE, OAuth, OIDC, mTLS : compatible avec les passerelles, WAF et IdP existants
Stabilité depuis la v1.0	Proto normatif unique, règles de dépréciation, négociation de version
Fonctions entreprise	Multi-tenant, cartes signées, carte étendue authentifiée, OAuth modernisé (PKCE, device code)
Gouvernance neutre	Linux Foundation puis AAIF, plus de 150 organisations, support natif chez les trois grands clouds
Écosystème	SDK officiels en Python, Go, Java, JavaScript, .NET, Rust ; Inspector et TCK pour la conformité
Complémentarité avec MCP	Répartition claire : MCP pour les outils, A2A pour les agents, sous la même fondation

Inconvénients

Inconvénient	Conséquence pratique
Complexité supérieure à un simple appel d'API	Pour un besoin déterministe et interne, un outil MCP ou une API REST est plus simple et plus rapide
Surcoût de latence et de tokens	Chaque saut ajoute un aller-retour réseau et souvent un appel LLM ; les chaînes profondes deviennent lentes et chères
Opacité à double tranchant	On ne voit pas comment l'agent distant raisonne : débogage, explicabilité et audit plus difficiles, dépendance à la qualité de ses messages et artifacts
Hétérogénéité des implémentations	Tous les frameworks ne couvrent pas tout (push, SubscribeToTask, carte étendue) : tester l'interopérabilité réelle
Sécurité largement à la charge de l'implémenteur	Le protocole déclare les schémas mais n'empêche ni SSRF, ni injection de prompt, ni sur-délégation

Inconvénient	Conséquence pratique
Webhooks exigeants	Endpoint public joignable, livraison au moins une fois, idempotence et vérification d'authenticité obligatoires

Limites structurelles (état v1.0)

1. **Pas d'API de registre standard** : chaque plateforme propose son catalogue, la découverte à l'échelle d'un écosystème reste propriétaire.
2. **Pas de schéma d'entrée ou de sortie par compétence** : les skills sont décrits en langage naturel ; les contrats de données machine à machine passent par conventions, extensions ou documentation.
3. **Délégation d'identité multi-sauts non spécifiée** : réduire la portée des jetons à chaque saut (token exchange, audience) est à construire soi-même.
4. **Fiabilité de livraison limitée** : les messages de statut ne sont pas garantis, les webhooks sont « au moins une fois » sans ordre global ; seuls l'état de la tâche et les artifacts sont une source de vérité.
5. **Pas de streaming bidirectionnel** : le client ne peut pas pousser des messages dans un flux ouvert pendant que l'agent travaille ; chantier prévu en v1.1.
6. **Retry, timeouts et disjoncteurs hors protocole** : laissés au client, donc hétérogènes.
7. **Pas de sémantique de coût, de quota ou de SLA** dans la carte : négociation hors bande.
8. **Confiance ≠ authenticité** : une carte signée prouve l'émetteur, pas la qualité ni l'honnêteté de l'agent ; les cadres de confiance (qui certifie qui) ne sont pas standardisés.
9. **Pas de transport événementiel standard** (Kafka, AMQP) : uniquement via binding personnalisé non interopérable ou pont webhook.
10. **Gestion du contexte partagé minimale** : `contextId` regroupe, mais le partage de mémoire ou d'état entre agents n'est pas défini, par conception.

Quand utiliser A2A, quand l'éviter

Situation	Recommandation
Agents d'équipes, de technologies ou d'éditeurs différents qui doivent coopérer	A2A
Tâches longues, multi-tours, avec validation humaine	A2A

Situation	Recommandation
Exposer un agent à des partenaires ou l'intégrer dans une plateforme SaaS	A2A, avec passerelle et carte signée
Appel d'une fonction déterministe (base, API, calcul)	MCP ou API classique
Plusieurs agents dans une même application et un même framework	Orchestration native du framework ; A2A seulement aux frontières
Latence critique de quelques millisecondes	Éviter les chaînes A2A ; appel direct
Flux de données massifs ou temps réel continu	Bus d'événements ou streaming dédié ; A2A pour le pilotage des tâches

11. Écosystème, feuille de route et ressources

La v1.0 est la cible à implémenter aujourd'hui ; la v1.1, en préparation selon la feuille de route mise à jour le 15 septembre 2026, vise le streaming bidirectionnel et la robustesse du cycle de vie des tâches.

Écosystème à date

Catégorie	Acteurs
Gouvernance	Linux Foundation, puis Agentic AI Foundation (projet « Growth Stage » depuis août 2026)
Clouds	Google Cloud, AWS Bedrock AgentCore Runtime, Microsoft Azure AI Foundry
SaaS	ServiceNow, Salesforce, Atlassian, SAP
Frameworks	LangGraph (Agent Server compatible v1.0), CrewAI, Pydantic AI, AG2, IBM BeeAI
SDK officiels	Python, Go, Java (dont A2A Jakarta), JavaScript (1.0 GA en juillet 2026), .NET, Rust
Passerelles	Plugins A2A dans des API gateways (Kong, Tyk) : routage, sécurité, métriques OpenTelemetry
Outillage	A2A Inspector, A2A TCK (conformité)

Feuille de route (v1.1 et au-delà)

- **Robustesse du protocole** : spécification plus fine de la chronologie des tâches, filtrage d'événements standardisé et rétrocompatible, règles claires pour les messages reçus pendant qu'une tâche est en `WORKING`.
- **Streaming bidirectionnel** : mises à jour d'artifacts en temps réel, gestion des rôles et messages multi-tours pendant l'exécution.
- **CLI A2A** et intégration aux environnements de développement assistés par agents.
- Piste communautaire : normalisation des registres et de la découverte avancée.

Parcours d'apprentissage conseillé

1. Lire les sections 2 à 4 de ce cours, puis la section 3 de la spécification (opérations).
2. Faire tourner l'exemple Helloworld des `a2a-samples`, l'inspecter avec A2A Inspector.
3. Implémenter l'agent de synthèse de la section 8 avec multi-tour et streaming.
4. Ajouter OAuth client credentials, une carte étendue, puis signer la carte.
5. Activer les notifications push avec un récepteur idempotent, puis le brancher sur un bus d'événements.
6. Construire un orchestrateur qui découvre deux agents via un registre et combine A2A et MCP.

Sources consultées

- [Spécification A2A, dernière version publiée 1.0.0](#)
- [Spécification v1.0.0 figée](#)
- [What's New in v1.0](#)
- [Agent Discovery](#)
- [Annonce : A2A rejoint l'Agentic AI Foundation, 27 août 2026](#)
- [Archive du blog A2A 2026 \(publication de la v1.0\)](#)
- [Feuille de route A2A](#)
- [Tutoriel Python : Agent Executor](#)
- [Tutoriel Python : démarrage du serveur](#)
- [Tutoriel Python : client](#)
- [Tutoriel Python : installation](#)
- [SDK JavaScript 1.0.0 \(notes de version\)](#)
- [A2A Jakarta 1.0.0.CR1](#)
- [LangChain Agent Server : endpoint A2A](#)
- [Tyk : architecture et limites pratiques d'A2A](#)

- [a2a.proto \(mappings REST, via a2a-types\)](#)
- [Kong : plugin AI A2A Proxy](#)